Technical Support & Monitoring System

A comprehensive manual for quality assurance auditing, bug resolution, and real-time network service monitoring.

Project Title:	Support Service Application
Student:	Yunus Abdullayev
Type:	Enterprise Support Portal
Version:	1.0.0
Date:	June 2026

Table of Contents

This user manual details the local installation process, database relational mappings, and step-by-step role guides.

1. Introduction & System Goals

The **Support Service Application** is a centralized corporate portal developed to manage customer service quality, catalog and resolve bugs across different software products (e.g. CRM, SIPUNI, SARKOR, MODERATOR), and monitor the uptime stability of external network APIs.

This system operates both as a **live monitoring agent** and a **helpdesk CRM**. It monitors the endpoints of critical company tools at set intervals, registers outages, opens system incidents automatically, and alerts engineers through a Telegram notification gateway.

Project Architecture:

- **Backend API:** Node.js, NestJS v10 (Prisma ORM, PostgreSQL database)
- **Frontend Client:** Next.js (App Router, Tailwind CSS, shadcn/ui components)
- **Referential Integrity:** Managed via PostgreSQL relations to ensure that bugs, comments, and reviews are mapped to corresponding user IDs and products.

> 🔒 **Security and Authorization**
>
> Passwords are encrypted using bcrypt hashing. Access tokens are generated using JWTs, ensuring all API endpoints validate headers for proper Role-Based Access Control (RBAC) levels.

2. Local Installation & Configuration

Follow these steps to deploy the application in a local development environment:

1. **Install Server Dependencies**

   Navigate to the NestJS API folder and run package installation:

```
cd backend
npm install
```

2. **Create Environment Config**

   Create a `.env` file in the `backend` root with the following parameters:

```
DATABASE_URL="postgresql://postgres:password@localhost:5432/supportdb?schema=public"
JWT_SECRET="support_app_secret_key_2026"
PORT=3005
```

3. **Migrate and Seed Ma'lumotlar Bazasi**

   Execute Prisma migrations and run database seeding scripts:

```
npx prisma migrate dev --name init
npx prisma db seed
```

   Launch the API server:

```
npm run start:dev
```

4. **Run Frontend Application**

   Navigate to the frontend folder, install react components, and start Next.js:

```
cd ../frontend
npm install
npm run dev
```

The interface is now accessible at `http://localhost:3000`.

3. User Roles (RBAC)

Users are restricted to operations designated to their authorization roles:

Role	System Permissions
ADMIN	Platform configuration. Can add/modify users, assign permissions, change active status, edit product lists, and configure the Telegram API credentials.
TEAM_LEADER	Quality Assurance. Conducts Dialog Reviews on operators, writes knowledge base articles, monitors service response metrics, and audits incidents.
OPERATOR	Client interaction. Registers bugs, upvotes existing bugs on customer calls, logs client complaints, and searches the Knowledge Base.
DEVELOPER	Issue resolution. Accesses assigned bugs and incidents, updates work progress, sets statuses (FIXED, CLOSED), and closes incidents with a root cause logs entry.

4. Core System Modules

4.1. Network Monitoring & Incidents

The Service Monitor regularly sends requests to configured external system endpoints and logs availability metrics.

- **Configurations:** Every monitor has an endpoint URL, HTTP method, check intervals, and expected response codes.
- **Monitor Logs:** Records latency (in milliseconds) and returned statuses for every ping.
- **Auto Incident Trigger:** When an endpoint fails to return the expected status code, the background cron task triggers a `CRITICAL` or `HIGH` incident, links the affected product, and sends alert payloads to the Telegram gateway. Engineers resolve the outage and log the root cause.

4.2. Bug Tracking & Reporting Flows

When clients report malfunctions, operators record the details in the centralized Bugs module.

Every Bug entry logs the following:

- **Title & Description:** Description of the issue and affected components.
- **Steps to Reproduce:** Steps developers must take to recreate the error.
- **Expected & Actual Results.**
- **Attachments:** File uploads for screenshots or error logs.
- **Multi-client Upvoting:** If multiple clients complain about the same bug, operators upvote the issue. This allows developer managers to identify and prioritize high-impact bug fixes.

4.3. Dialog Quality Reviews (Operator Audits)

Team leaders evaluate chat logs of support operators to ensure optimal customer communication quality.

The rating metrics (each out of 100 points):

- **First Response Score:** Speed of response and greeting tone.
- **Understanding Score:** Accurate capture of customer issues.
- **Communication Score:** Professional language and attitude.
- **Closing Score:** Proper closure and satisfaction questions.

- **Solution Score:** Competency of the technical help provided.

- **Total Score:** The average of all metrics, computed automatically.

4.4. Improvement Requests Management

Tracks client feature requests and product enhancement ideas:

- **Business Value:** The estimated value of the feature for customer retention.
- **Otvotes:** Keeps track of how many clients request the same feature.
- **Status Pipeline:** Moves requests from `NEW` to `UNDER_REVIEW`, `PLANNED`, `IN_DEVELOPMENT`, `DONE`, or `REJECTED`.

4.5. Knowledge Base & Categorized Articles

Enables operators to access standard operating procedures (SOPs) and technical solutions quickly.

Categories include:

- **CRM and SIPUNI:** Guidelines for telephony issues and database tasks.
- **SARKOR and MODERATOR:** Technical specs of internal platforms.
- **Troubleshooting:** Standard solutions to recurring technical issues.

4.6. Corporate Clients & Comments Logging

Maintains directory profiles of corporate clients, noting branch counts, employee details, and specific operational comments. This context populates on-screen when a corporate client requests support.

4.7. Client Difficulties & Upvoting

Documents components of products where users struggle the most. Upvotes indicate usability bottlenecks, which product managers analyze to schedule user interface improvements.

5. Telegram Notification System Setup

The platform is equipped with a Telegram Bot integration to send real-time warning payloads to system administrators during outages:

1. **Bot Token Input:** Under Admin panel `Telegram Settings`, input your Telegram Bot API Token.
2. **Phone Number Linking:** Link phone numbers of responsible staff members.
3. **Chat ID Routing:** The system maps incoming alerts to linked user accounts via their unique `telegramChatId` variables to ensure alerts target correct developer chats.

6. Troubleshooting FAQ

Q: How does the network monitoring process work automatically?
A: NestJS implements a background cron job using `@nestjs/schedule`. This cron job runs at regular intervals to ping endpoints and log response values.

Q: Can an operator configure service monitors?
A: No. Managing, adding, or editing monitors is restricted to `ADMIN` role holders.

Q: How is the average Dialog Review score calculated?
A: When a Team Leader submits scores for the five metrics, the system calculates the arithmetic average and updates the database `totalScore` property.

Q: How do we reset the database to default seeded users?
A: In the backend folder, execute `npx prisma db seed` to repopulate all database models with default test records.